SIT720 - Machine Learning

Task 4.1P: Clustering and Principal Component Analysis

Report

Student name: Zenith Anthony

Student ID: s225534396

Date: _____

Completion status: Both datasets were executed in the accompanying notebook. Questions 1-5 use the supplied Obesity dataset, and Questions 6-8 use the supplied UCI Gene Expression Cancer RNA-Seq dataset. All numerical values and figures reported below are based on the executed notebook outputs.

Part A - Lesson Review and Reflection

A1. Key learning from Week 3: Clustering and unsupervised learning

Two important Week 3 concepts are K-Means clustering and the Silhouette Coefficient. K-Means is an unsupervised method that divides observations into k groups by repeatedly assigning each observation to the nearest centroid and then updating each centroid to the mean of its assigned points. The process reduces within-cluster squared distance. It is important because many datasets do not contain labels, yet analysts still need to discover structure. For example, a health service could cluster patients using activity, diet and body measurements to identify groups with similar behaviour patterns for targeted interventions.

The Silhouette Coefficient evaluates how well each observation fits its own cluster compared with the nearest alternative cluster. Values close to 1 indicate compact, well-separated clusters; values around 0 indicate overlap; negative values suggest possible misassignment. It is useful because K-Means requires k to be selected before training. In this task I compare k=2 to 10 and choose the value with the largest average silhouette. This gives a data-driven choice rather than selecting k only from the known class labels.

A2. Key learning from Week 4: Dimensionality reduction and PCA

Principal Component Analysis (PCA) is a dimensionality-reduction method that transforms the original variables into new, mutually orthogonal directions called principal components. PC1 captures the largest possible variance, PC2 captures the largest remaining variance subject to being orthogonal to PC1, and the process continues for later components. PCA is important in machine learning because high-dimensional data can be slow to process, difficult to visualise and affected by redundant or highly correlated variables. A practical example is gene-expression analysis: tens of thousands of gene measurements can be projected into three components to visualise whether tumour samples form visible groups.

Explained variance describes how much of the dataset's total variance is represented by each principal component. The explained-variance ratio is the eigenvalue for a component divided by the sum of all component eigenvalues. It helps determine whether the reduced representation retains enough information for the intended task. If three components explain only a small percentage, they may still be useful for visualisation but may lose information that matters for clustering or prediction.

A3. Applying clustering and PCA to data analytics

Clustering can uncover groups that are not visible from individual variables. In a health dataset, observations may cluster according to combinations of diet, activity, body measurements and habits even when no label is supplied. This can reveal behavioural profiles or subpopulations that support exploratory analysis. However, clustering is sensitive to scale, encoding and the assumed geometry of the algorithm, so the discovered groups should be interpreted rather than treated automatically as true classes.

Dimensionality reduction helps when many features are redundant or when distances become difficult to interpret in very high dimensions. PCA can compress correlated information, reduce computation and create two- or three-dimensional views that make structure easier to inspect. A strong application is cancer gene-expression analysis. PCA can first summarise thousands of gene measurements, after which K-Means can be applied to search for tumour subgroups. The clustering results can then be compared with known tumour labels. Using the two methods together therefore combines visual exploration, computational efficiency and quantitative validation, while also exposing the trade-off between compression and information preservation.

Part B - Problem Solving Practical Task

1. Data preparation and encoding

The uploaded Obesity dataset contains 2,111 observations, 16 input features and the class label `NObeyesdad`. No missing values were detected. The target has seven classes. For clustering, I removed the target before preprocessing so no class information could leak into the unsupervised model. After encoding, the feature matrix contains 20 numerical columns.

Variable(s)	Type	Transformation / encoding	Reason
Gender	Binary categorical	Female=0, Male=1	Two categories can be represented with one binary dimension.
family_history_with_overweight, FAVC, SMOKE, SCC	Binary yes/no	no=0, yes=1	Preserves the two-state meaning without adding redundant columns.
CAEC, CALC	Ordinal frequency	no=0, Sometimes=1, Frequently=2, Always=3	The categories have a natural increasing frequency order.
MTRANS	Nominal categorical	One-hot encoding	Transport categories have no meaningful numeric order.
Continuous/integer variables	Numeric	Kept numeric; then standardized	Standardisation prevents large-scale features from dominating Euclidean distance.

Encoding changes clustering geometry. One-hot encoding avoids false ordering but increases dimensionality and can give a categorical feature several distance dimensions. Ordinal encoding is compact and preserves order, but it assumes the steps between categories are meaningful. I therefore use a mixed strategy based on the semantic type of each variable, followed by StandardScaler for the full encoded matrix.

```
X['Gender'] = X['Gender'].map({'Female':0, 'Male':1})
for c in ['family_history_with_overweight','FAVC','SMOKE','SCC']:
    X[c] = X[c].map({'no':0, 'yes':1})
X['CAEC'] = X['CAEC'].map(ordinal_map)
X['CALC'] = X['CALC'].map(ordinal_map)
X = pd.get_dummies(X, columns=['MTRANS'], dtype=int)
X_scaled = StandardScaler().fit_transform(X)
```

2. Determining the optimal number of clusters

I fitted K-Means++ models for k=2 through 10 using 20 restarts for each k and calculated the Silhouette Coefficient. The maximum silhouette value is the selection criterion because it rewards high within-cluster cohesion and separation from neighbouring clusters.

k	Silhouette	Calinski-Harabasz	Davies-Bouldin
2	0.225	275.5	2.305
3	0.179	254.5	2.356
4	0.197	232.7	1.977
5	0.193	218.6	1.683
6	0.122	218.2	1.666
7	0.141	216.5	1.647
8	0.144	220.8	1.633
9	0.165	218.6	1.628
10	0.169	218.4	1.532

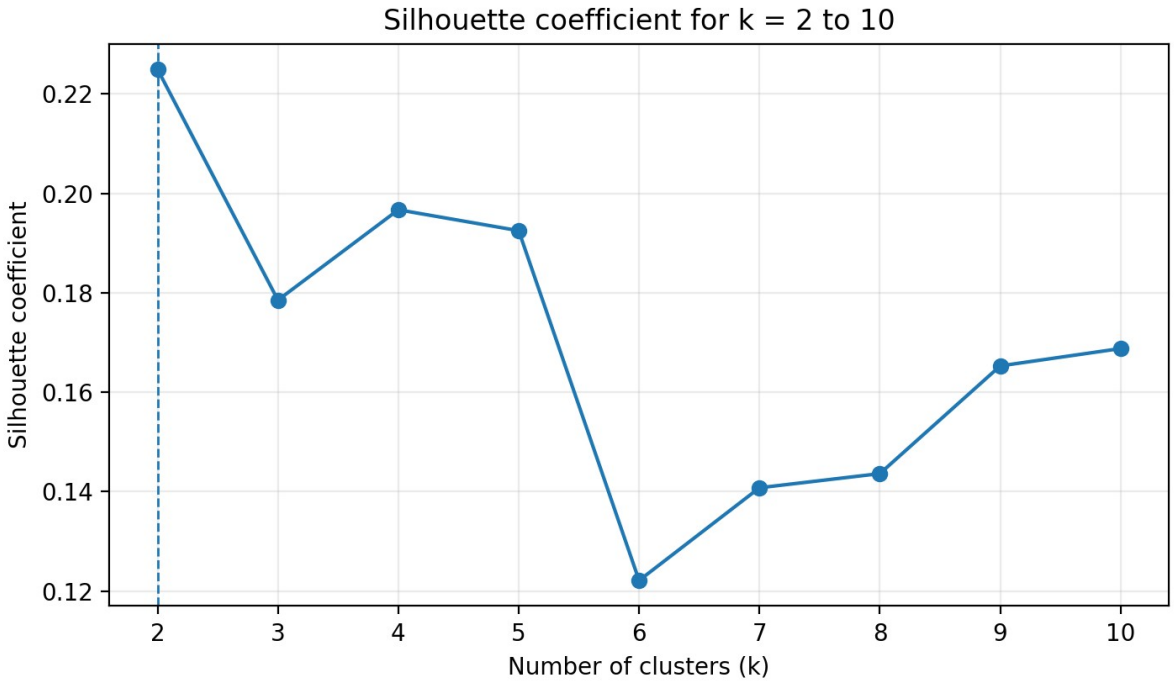


Figure 1. Silhouette coefficient across k=2 to 10.

The optimal value is k=2, with a silhouette coefficient of 0.225. This is different from the seven supplied obesity classes. The disagreement is not an error: K-Means optimises compactness in the encoded feature space, whereas `NObeyesdad` is an externally provided outcome class. The UCI documentation also notes that the dataset combines collected and synthetically generated records. Seven labelled categories therefore do not have to appear as seven well-separated spherical groups. The relatively modest silhouette values for most k values provide quantitative evidence of overlap between potential clusters.

3. Comparing K-Means and K-Means++

Using k=2, I compared random centroid initialization with K-Means++ while keeping the same seed, maximum iterations and a single start (`n_init=1`). A single start is deliberate here because it

exposes the effect of the initialization strategy; the next question then tests stability over many starts.

Method	Sil.	CH	DB	ARI	NMI	Inertia	Iter.
K-Means (random init)	0.146	213.5	2.871	0.096	0.168	38339.4	7
K-Means++	0.159	214.0	2.845	0.093	0.177	38331.1	12

For seed 42, K-Means++ achieved silhouette 0.159 compared with 0.146 for random initialization, and its inertia was slightly lower (38331.1 versus 38339.4). Its Davies-Bouldin value was also slightly lower, which is desirable. The ARI difference was very small and slightly favoured the random-start model. Therefore K-Means++ improved several internal metrics in this run but did not dominate every metric. K-Means++ selects dispersed initial centers, reducing the chance of poor starting centroids, but the iterative algorithm can still converge to different local minima and there is no guarantee that every K-Means++ run will be best.

4. Stability analysis of K-Means

I repeated both initialization strategies 50 times with seeds 0-49 and one start per seed. The table reports mean +/- sample standard deviation. A smaller standard deviation indicates more consistent outcomes across different starts.

Method	Silhouette	CH	DB	ARI	NMI	Inertia
K-Means (random init)	0.223 +/- 0.097	217.3 +/- 52.5	2.353 +/- 0.587	0.047 +/- 0.041	0.098 +/- 0.072	38295.7 +/- 876.5
K-Means++	0.218 +/- 0.095	220.7 +/- 52.2	2.352 +/- 0.571	0.044 +/- 0.040	0.093 +/- 0.070	38240.2 +/- 871.2

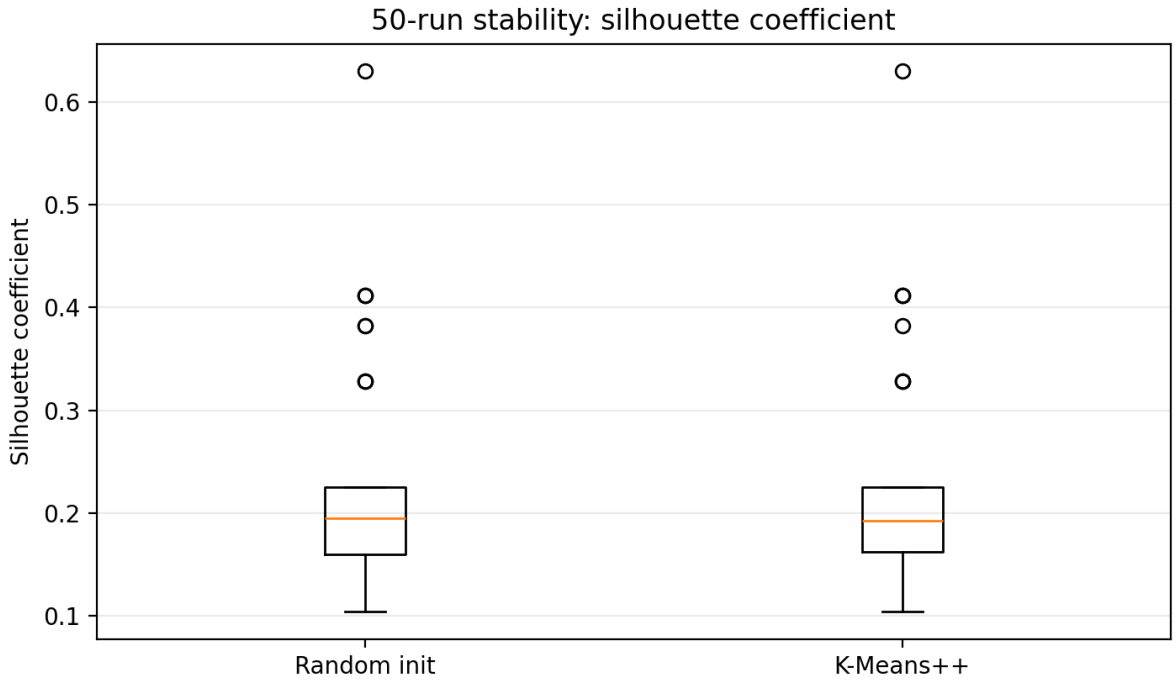


Figure 2. Distribution of silhouette scores over 50 independent starts.

The random-initialized K-Means mean silhouette was 0.223 (SD 0.097), while K-Means++ averaged 0.218 (SD 0.095). K-Means++ had slightly lower mean inertia (38240.2 vs 38295.7), but the variability of both methods remained similar. The single seed-42 results in Question 3 are below the 50-run

mean silhouette for both methods, showing why one run is not enough to judge stability. In this dataset K-Means++ is not consistently superior across all measures when only one start is allowed; using multiple restarts is still recommended.

5. Density-based clustering with DBSCAN

DBSCAN requires `eps` and `min_samples`. I used `min_samples=20`, approximately the encoded dimensionality (20), so a core point must have a substantial local neighbourhood. I then plotted the sorted distance to each observation's 20th nearest neighbour. The curve bends upward at roughly 6, so I selected `eps=6.0`. This combines a dimensionality-based neighbourhood rule with a data-driven k-distance inspection.

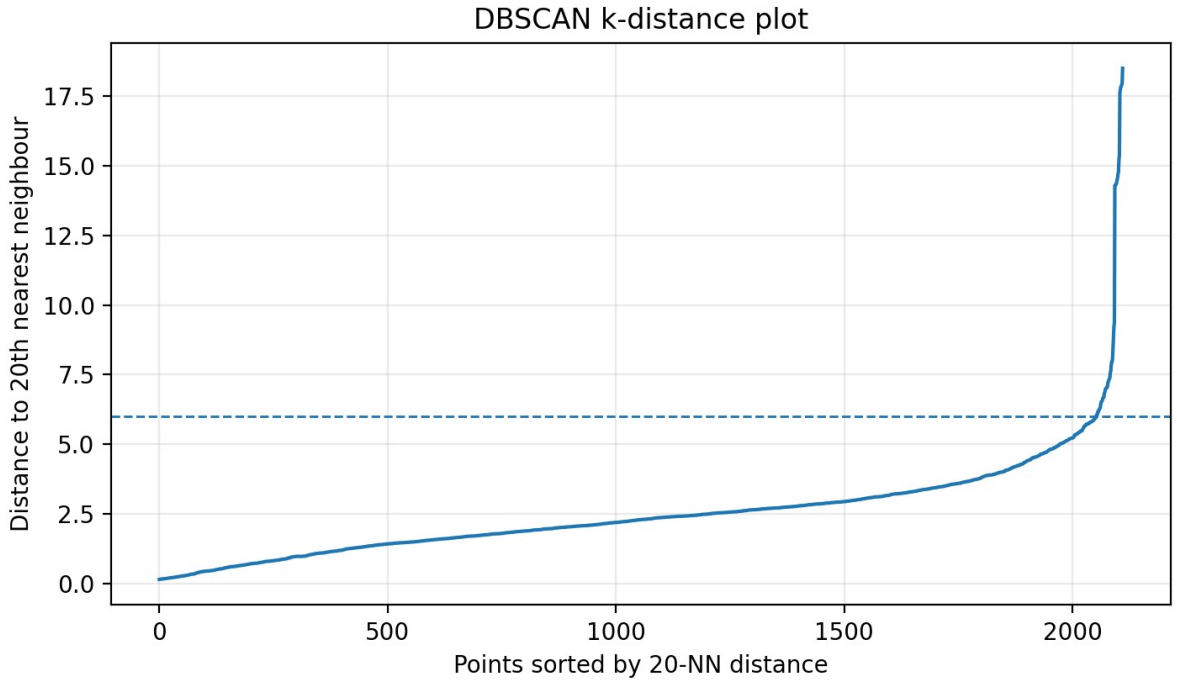


Figure 3. 20-nearest-neighbour distance curve used to select eps.

Method	Clusters	Noise pts	Noise %	Silhouette	ARI	NMI
K-Means++	2	0	N/A	0.159	0.093	0.177
DBSCAN	3	29	1.37%	0.394*	0.003	0.032

*DBSCAN silhouette is calculated on non-noise observations only.

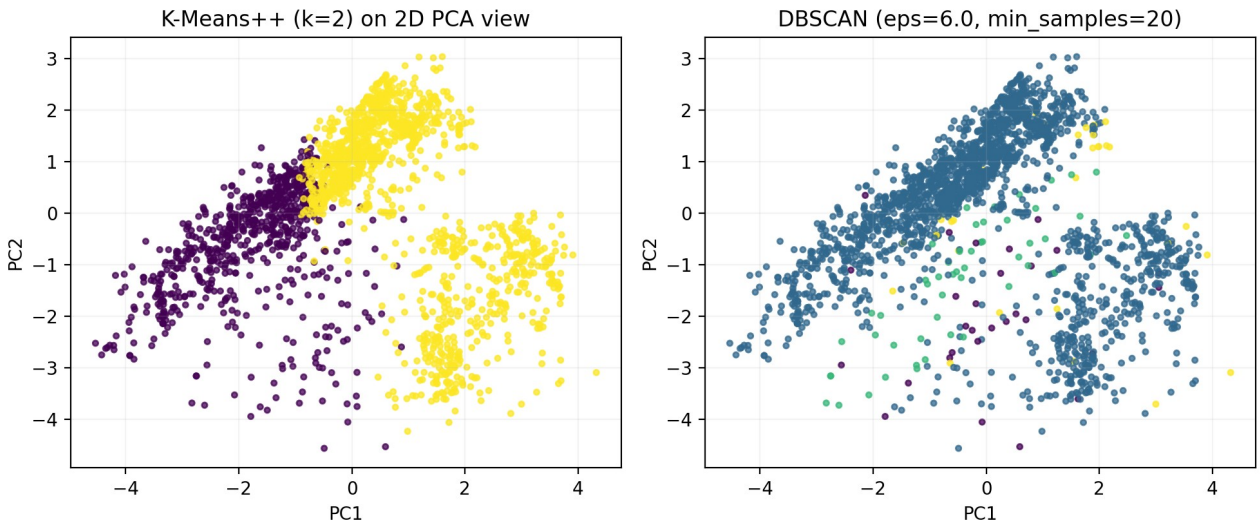


Figure 4. Two-dimensional PCA view of K-Means++ and DBSCAN assignments (PCA used only for display).

DBSCAN produced 3 clusters and 29 noise points (1.37% of the data). The largest cluster contains 1996 observations, while the smaller dense groups contain 51 and 35 observations. This is structurally different from K-Means, which assigns every point to one of exactly k centroids. DBSCAN can represent irregular dense regions and explicitly flag outliers; however, its output is sensitive to scaling, `eps`, `min_samples` and varying densities. For this dataset the ARI/NMI agreement with the seven obesity labels is very low, so the density structure should not be interpreted as reproducing the supplied classes. K-Means is simpler and gives a fixed partition, while DBSCAN is more useful when noise detection or non-spherical structure is important.

6. Principal Component Analysis of Gene Expression Data

The gene-expression dataset contains 801 tumour samples and 20,531 numeric gene-expression features. The supplied labels contain five tumour classes: BRCA (300), KIRC (146), LUAD (141), PRAD (136) and COAD (78). No missing feature values were found. I removed the sample identifier, standardized every gene feature to zero mean and unit variance, and applied PCA to generate three principal components.

```
gene_X_scaled = StandardScaler().fit_transform(gene_X)
pca3 = PCA(n_components=3, random_state=42)
gene_pcs = pca3.fit_transform(gene_X_scaled)
```

The 3D PCA plot shows meaningful tumour-related structure even though PCA is unsupervised. KIRC and PRAD form relatively distinct groups, COAD is also fairly compact, while BRCA occupies a broader region and LUAD overlaps more with neighbouring groups in the three-component space. This demonstrates that the leading components preserve biologically relevant variation, but the visible overlap also indicates that three components do not preserve all class-separating information.

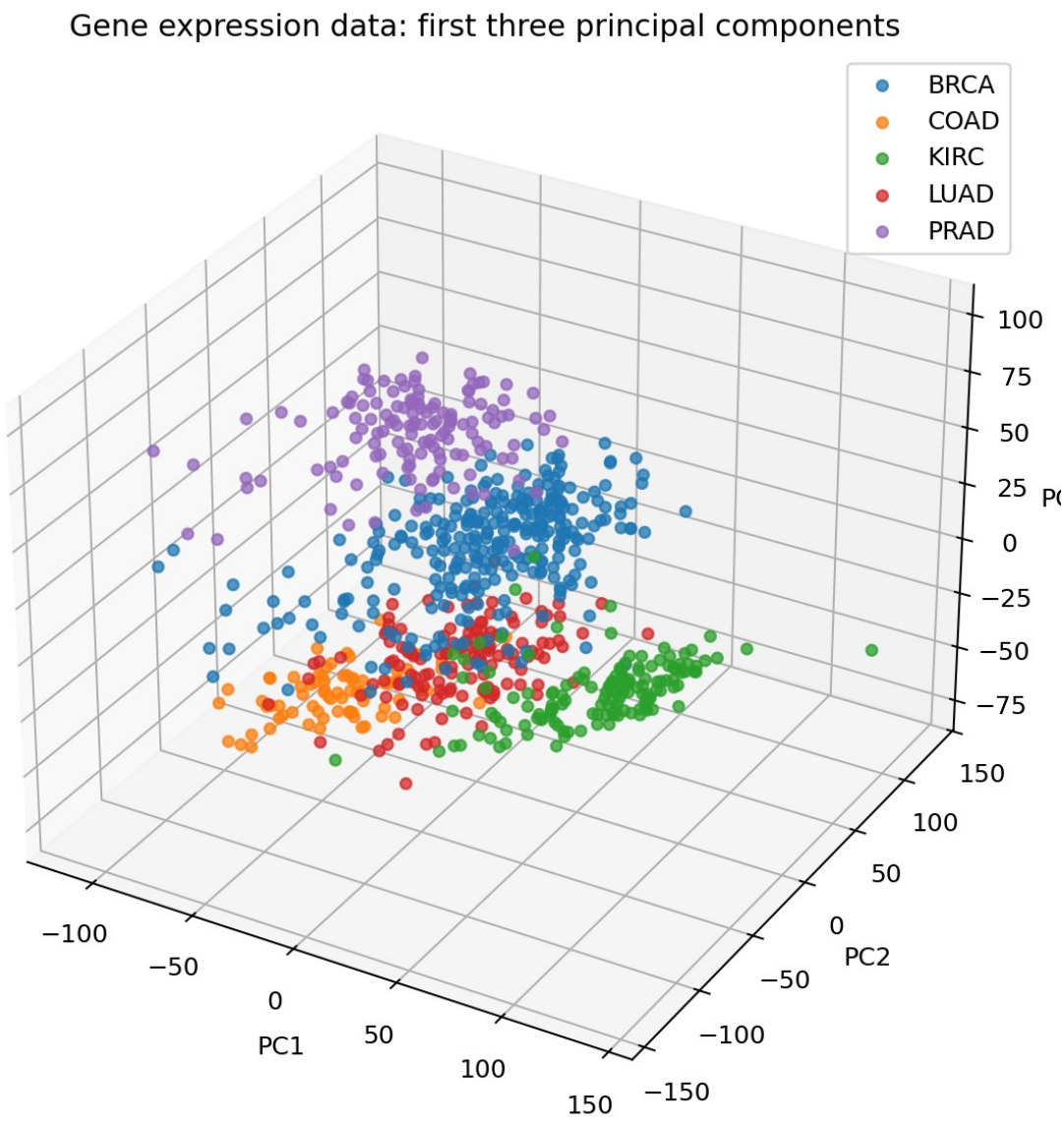


Figure 5. First three principal components of the gene-expression dataset, coloured by supplied tumour class.

7. Understanding variance explained by PCA

For each principal component, PCA computes an eigenvalue representing the variance captured along that direction. The explained-variance ratio is eigenvalue_i divided by the sum of all eigenvalues. Using standardized gene features, PC1 explains 10.540% of total variance, PC2 explains 8.754%, and PC3 explains 7.810%. Their cumulative explained variance is 27.104%.

Therefore, the first three components retain only about 27.1% of the total standardized-data variance. This is adequate for a compact 3D visualisation of broad structure, but it is not an adequate replacement for the full feature set when the goal is to preserve most of the information. More components would be required for a higher-fidelity representation.

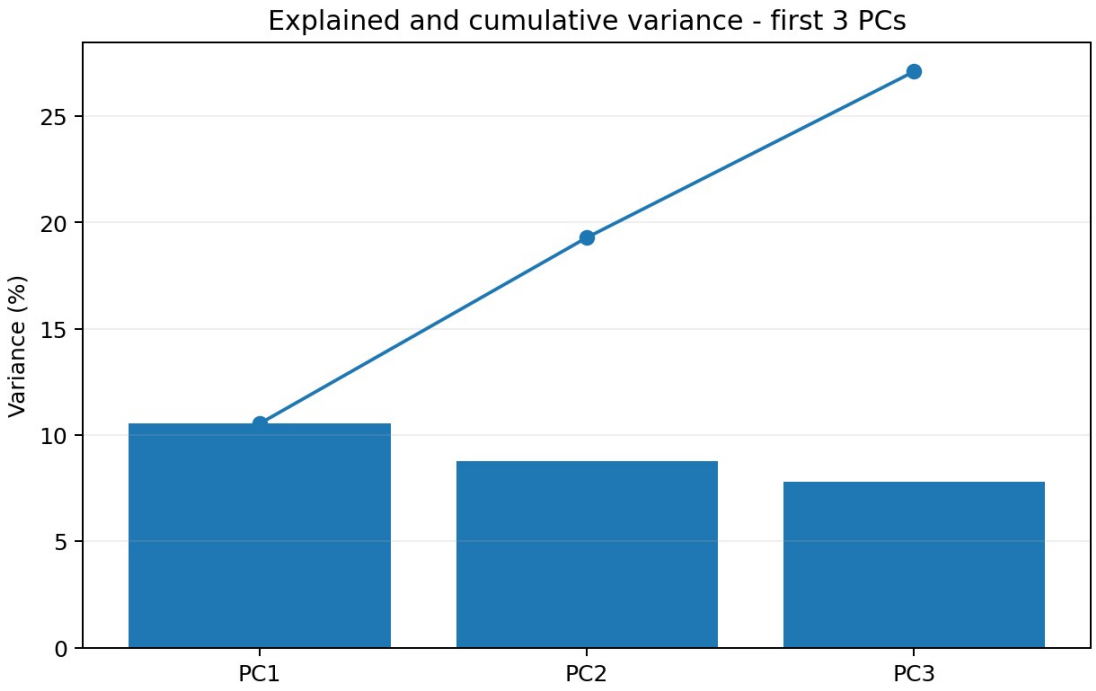


Figure 6. Explained and cumulative variance for PC1-PC3.

8. Evaluating PCA for clustering

I fitted K-Means with k=5 to (1) the standardized original 20,531-dimensional gene matrix and (2) the three-dimensional PCA representation. Both models used K-Means++ initialization, 20 restarts, maximum 500 iterations and random_state=42. The supplied tumour labels were used only for external evaluation with ARI, NMI, homogeneity, completeness and V-measure.

> The original standardized feature space performed substantially better: ARI=0.7941, NMI=0.8539, homogeneity=0.8610, completeness=0.8469 and V-measure=0.8539. With only the first three PCs, performance fell to ARI=0.6060, NMI=0.6726, homogeneity=0.6887, completeness=0.6573 and V-measure=0.6726.

Representation	ARI	NMI	Homogeneity	Completeness	V-measure
Original genes	0.7941	0.8539	0.8610	0.8469	0.8539
First 3 PCs	0.6060	0.6726	0.6887	0.6573	0.6726

Table 6. External clustering evaluation against the supplied tumour labels.



Figure 7. K-Means clustering quality on the original standardized genes and the first three PCs.

The contingency tables support the same conclusion. In the original feature space, KIRC (144/146), PRAD (134/136), LUAD (139/141) and COAD (74/78) are each concentrated strongly in individual clusters, although BRCA is split mainly across two clusters. In the three-PC representation, BRCA is more fragmented and several classes show greater mixing. The reduction to three components has therefore removed some class-relevant information.

> PCA is useful for visualisation and can reduce computational cost, but I would not recommend using only three PCs as the preprocessing representation for K-Means on this dataset when clustering quality is the priority. The drop in ARI from 0.7941 to 0.6060 and NMI from 0.8539 to 0.6726 is substantial. A better approach would be to retain more principal components and select their number using cumulative explained variance and clustering validation.

Conclusion

The Obesity analysis shows that preprocessing decisions strongly influence distance-based learning. After mixed categorical encoding and scaling, the Silhouette Coefficient selected k=2 rather than the seven supplied obesity labels, demonstrating that unsupervised structure and known classes can differ. K-Means++ reduced inertia slightly but did not remove run-to-run variability, while DBSCAN found three dense groups and 29 noise points. For the gene-expression data, the first three PCs explained 27.104% of total variance and provided a useful 3D view of tumour structure. However, K-Means on the original standardized genes achieved ARI=0.7941 and NMI=0.8539, compared with ARI=0.6060 and NMI=0.6726 after reduction to three PCs. Thus, PCA is valuable for visualisation here, but three components sacrifice too much information for the best clustering performance.

Generative AI acknowledgement

I used ChatGPT (OpenAI) to assist with planning the analysis, generating and debugging Python code, interpreting the numerical outputs, and editing the report for clarity. The numerical results and figures reported in this submission were produced by executing the notebook on the supplied Obesity dataset and the UCI Gene Expression Cancer RNA-Seq dataset. I used the GenAI output as support for completing the analysis and understanding the methods rather than as a substitute for the required machine-learning work.

References

Deakin University. (2026). SIT720 Task 4.1P: Clustering and PCA [Assessment brief].

UCI Machine Learning Repository. (2019). Estimation of Obesity Levels Based On Eating Habits and Physical Condition [Dataset]. https://doi.org/10.24432/C5H31Z

Fiorini, S. (2016). Gene Expression Cancer RNA-Seq [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5R88H

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.